

WorkFlow AI

Product Requirements Document

“Turn workplace information into action.”

Document status	Draft — MVP / hackathon build
Version	1.0
Last updated	2026-09-12
Product	WorkFlow AI — AI workplace document copilot

Table of Contents

Table of Contents	2
1. Overview	4
1.1 Document Purpose	4
2. Problem Statement	4
3. Goals & Objectives	4
4. Target Users & Personas	5
5. Scope	5
5.1 In Scope (current build)	5
5.2 Out of Scope (current build)	6
6. Features & Functional Requirements	6
6.1 Workspace Access	6
6.2 Document Management	6
6.3 AI Copilot (Retrieval-Augmented Q&A)	7
6.4 Document Intelligence: Summarization & Action Extraction	7
6.5 Action Center & Communications	7
6.6 AI Reports	8
6.7 Spreadsheet Analysis	8
6.8 Analytics Dashboard	8
6.9 Settings & Workspace Management	8
7. Non-Functional Requirements	9
8. System Architecture Overview	9
8.1 High-Level Components	9
8.2 Core Data Flow	10
9. Data Model	10
10. API Surface (Summary)	11
11. AI / RAG Pipeline Design	12
11.1 Embeddings	12
11.2 Map-Reduce Document Processing	12
11.3 Retrieval-Based Report Generation	12
11.4 Vector Storage & Retrieval	12

11.5 Grounding & Fallback Behaviour	13
12. Success Metrics / KPIs	13
13. Constraints, Assumptions & Dependencies	13
13.1 Constraints.....	13
13.2 Assumptions	13
13.3 Dependencies	14
14. Risks & Mitigations	14
15. Release Status & Open Items	14
15.1 Open items before a production launch	15
16. Appendix: Glossary	15

1. Overview

WorkFlow AI is a workplace document intelligence platform. Teams upload the documents they already produce — maintenance reports, meeting minutes, safety procedures, management briefs, spreadsheets — and the product turns that unstructured information into things a team can act on: grounded answers to questions, extracted action items with owners and deadlines, generated communications, and structured reports.

The product is built around a retrieval-augmented generation (RAG) pipeline: documents are parsed, chunked, embedded, and stored in a vector database; user questions are answered by retrieving the most relevant passages and grounding an LLM's response in them, with inline citations back to the source document and page.

1.1 Document Purpose

This PRD defines what WorkFlow AI does, who it is for, the functional and non-functional requirements of the current build, the underlying system architecture (including the AI/RAG pipeline design), and the known constraints and open work. It reflects the state of the implementation as of this document's version, including a recent overhaul of the retrieval layer (real semantic embeddings, map-reduce document processing, and retrieval-based report generation — see Section 11).

2. Problem Statement

Operational teams generate large volumes of documents — inspection reports, meeting minutes, safety procedures, spreadsheets of incidents or metrics — but that information stays locked in files. Finding an answer means manually searching PDFs; turning a report into follow-up work means someone reading it end-to-end and writing down action items by hand; producing a summary or a risk brief for management means re-reading and re-writing the same source material. This is slow, inconsistent, and easy to get wrong when a document is long or a critical detail is buried on a later page.

WorkFlow AI addresses this by making uploaded documents queryable in natural language, and by automating the repetitive parts of turning a document into action — summarization, action-item extraction, communication drafting, and report generation — while keeping every output traceable to a specific source document and page.

3. Goals & Objectives

- Let a user ask natural-language questions over their uploaded documents and get answers grounded in and cited to the source material.

- Automatically extract actionable items (owner, deadline, priority) from documents so nothing buried in a long report gets missed.
- Reduce the time it takes to go from “raw document” to “usable output” (summary, action list, report, communication draft).
- Support the common document types operational teams actually produce: PDF, Word (DOCX), Excel (XLSX), and CSV.
- Work multilingually at the retrieval layer, not just in English.
- Be safely and cheaply demoable — fully functional with no external database configured — while also having a production path (Supabase Postgres + pgvector + Auth + Storage) for real deployments.

4. Target Users & Personas

Persona	Role	Needs from WorkFlow AI
Operations / Maintenance Lead	Reviews weekly maintenance and inspection reports across sites	Ask questions across many reports instead of re-reading them; get a risk assessment that doesn't miss issues buried deep in a report.
Site Safety Officer	Owns safety procedures and incident logs	Extract action items with deadlines from safety procedures and incident reports; draft follow-up communications to owners.
Project / Program Manager	Runs meetings, tracks decisions and follow-ups	Turn meeting minutes into a tracked action list automatically; generate management briefs for stakeholders.
Management / Executive	Consumes summarized information, not raw documents	Executive summaries and risk assessments generated on demand from the underlying operational documents.

5. Scope

5.1 In Scope (current build)

- Document upload and parsing for PDF, DOCX, XLSX, and CSV.
- Chunking, embedding, and vector storage of document content.
- Retrieval-augmented Q&A (“Copilot”) over one or more uploaded documents, with source citations.
- AI-generated document summaries and extracted action items.
- Action Center: tracked action items with status, priority, owner, deadline, and one-click communication drafting (email / WhatsApp).

- AI-generated structured reports (six report types — see Section 6.6) across one or more selected documents.
- Ad-hoc spreadsheet analysis (XLSX/CSV) with computed statistics and AI-written observations.
- Analytics dashboard: documents processed, questions answered, actions extracted, reports generated, and estimated time saved.
- A fully functional in-memory demo mode requiring no external database, alongside a production mode backed by Supabase.

5.2 Out of Scope (current build)

- Real-time multi-user collaboration on the same workspace.
- Editing documents in place (the product is read/analyze-only over uploaded files).
- Automated delivery of generated communications (drafts are generated for a human to review and send; there is no outbound email/WhatsApp integration).
- Fine-tuning or hosting a custom LLM (the product calls a hosted Groq model over its API).
- Support for file types beyond PDF, DOCX, XLSX, and CSV.
- Formal accessibility (WCAG) and internationalized UI text (the retrieval layer is multilingual; the interface itself is English-only).

6. Features & Functional Requirements

6.1 Workspace Access

Users enter the product either via a Supabase-authenticated account (production mode) or via a local “Explore Demo” guest session requiring no signup (demo mode). Backend endpoints are protected by bearer JWT verification against Supabase in production; in demo mode every request is transparently attributed to a fixed demo user so the product is explorable with zero configuration.

- FR-1.1: A user can start a guest session with one click and land on the dashboard with no credentials.
- FR-1.2: A user can sign up / log in with a real account when Supabase auth is configured.
- FR-1.3: All data (documents, actions, conversations, reports) is scoped to the authenticated user; Postgres Row Level Security is the isolation boundary in production.

6.2 Document Management

- FR-2.1: A user can upload a PDF, DOCX, XLSX, or CSV file up to 10 MB.
- FR-2.2: On upload, the system parses the file into pages, chunks the text (~900 characters per chunk with overlap), generates a vector embedding per chunk, and stores everything with the document's status moving uploading → processing → ready (or failed with an error message).

- FR-2.3: A user can view a list of their documents with status, page count, and chunk count, and can delete a document (removing its file and all derived chunks).
- FR-2.4: A user can view a document's extracted page-by-page text in-app without needing the original file.
- FR-2.5: The system enforces a per-workspace document limit (5 documents in the current demo-protection configuration).

6.3 AI Copilot (Retrieval-Augmented Q&A)

The Copilot lets a user ask natural-language questions and receive an answer grounded in their uploaded documents, optionally scoped to a single document. See Section 11 for the retrieval design.

- FR-3.1: A user can ask a question in a conversation; the system retrieves the most relevant passages from the workspace (or a chosen document) and generates an answer that cites passages inline (e.g. “[1]”) linked to filename and page number.
- FR-3.2: If no passage clears the relevance threshold, the system tells the user it could not find enough information rather than guessing.
- FR-3.3: Conversations persist and can be revisited; each message stores its cited sources alongside the answer text.
- FR-3.4: The system enforces a daily question limit (20/day in the current demo-protection configuration).

6.4 Document Intelligence: Summarization & Action Extraction

- FR-4.1: A user can request an AI summary of a ready document, returning a short overview plus 5–8 key points.
- FR-4.2: A user can request action-item extraction from a ready document, returning a de-duplicated list of concrete actions with title, description, suggested owner, deadline (if stated), and priority (low/medium/high/critical).
- FR-4.3: Both operations must process the entire document, not only its first few pages — see the map-reduce design in Section 11.2.

6.5 Action Center & Communications

- FR-5.1: A user can view all actions across the workspace (whether manually created or extracted from a document), with filtering and sorting by priority/status.
- FR-5.2: A user can create an action manually, or update an existing action's status, owner, deadline, or priority.
- FR-5.3: A user can generate a ready-to-send communication draft (email or WhatsApp message) for any action, referencing its source document when one exists.

6.6 AI Reports

A user selects one or more documents and a report type, and the system generates a structured Markdown report grounded in those documents (see Section 11.3 for how source content is selected).

Report type	Purpose
Executive Summary	High-level overview of outcomes and status for a non-technical audience.
Technical Report	Specifications, methodology, measurements, and equipment-level detail.
Management Brief	Decisions, priorities, budget, and timeline framing for management.
Risk Assessment	Hazards, critical issues, likelihood and impact, sourced across the selected documents.
Meeting Summary	Decisions made, action items, and discussion points from meeting-style documents.
Document Comparison	Differences and consistency across multiple selected documents.

- FR-6.1: Every generated report is saved to the workspace's report history and can be revisited.
- FR-6.2: The system enforces a daily report-generation limit (5/day in the current demo-protection configuration).

6.7 Spreadsheet Analysis

- FR-7.1: A user can upload an XLSX or CSV file for one-off, non-persisted statistical analysis (row/column counts, per-column min/max/mean, missing values) plus 4–6 AI-written plain-language observations about trends and anomalies, computed by pandas and narrated (not recalculated) by the LLM.

6.8 Analytics Dashboard

- FR-8.1: A user can see workspace-level counters: documents processed, questions answered, actions extracted, reports generated, and estimated minutes saved.
- FR-8.2: A user can see trends over time (documents processed, time saved) and a breakdown of actions by priority.
- FR-8.3: The dashboard surfaces current usage against the daily/workspace limits (documents, chat questions, reports).

6.9 Settings & Workspace Management

- FR-9.1: A user can see their profile info, current usage counters, and whether the backend is running against live services (Groq, Supabase) or in demo mode.
- FR-9.2: A user can reset their demo workspace back to a seeded, populated state for demonstration purposes.

7. Non-Functional Requirements

Category	Requirement
Security	Documents and derived data are isolated per user (Postgres Row Level Security in production); the LLM provider key and database service-role key are server-side only and never exposed to the browser; uploaded files are stored in a private bucket accessed only via short-lived signed URLs.
Reliability	Transient LLM API failures are retried with exponential backoff; if the LLM is unreachable or not configured, every AI feature falls back to a deterministic, content-grounded (non-LLM) response rather than failing outright.
Performance	Chat answers and document actions should return within the timeout budget of a single Groq call (60s) for single-pass features; multi-page documents processed via map-reduce take proportionally longer (multiple sequential LLM calls) — see Section 11.2.
Scalability	Vector search is delegated to Postgres/pgvector with an approximate-nearest-neighbour (ivfflat) index so query latency stays practical as chunk volume grows; the embedding model runs locally in the API process, avoiding a per-call external dependency.
Portability	The system runs with zero external services configured (full in-memory demo mode) or against a real Supabase project, without code changes — only environment configuration differs.
Cost	Embeddings are computed locally (no per-call API cost); the only metered external dependency is the Groq LLM API.
Usability	The interface must remain usable on both desktop and mobile widths (a dedicated mobile bottom navigation is provided).

8. System Architecture Overview

8.1 High-Level Components

- Frontend: Next.js 15 (App Router) / React 19 / TypeScript, styled with Tailwind CSS and Radix UI primitives, data-fetched via TanStack Query, charts via Recharts, authentication via Supabase's browser SSR client.
- Backend: FastAPI (Python), exposing a versioned REST API under `/api/v1`, running document parsing, the RAG pipeline, and all LLM orchestration.
- LLM: Groq-hosted “openai/gpt-oss-120b”, called through an OpenAI-compatible chat-completions interface (JSON mode used for structured outputs).
- Embeddings: intfloat/multilingual-e5-small, run locally inside the FastAPI process via sentence-transformers (no external embedding API) — see Section 11.1.

- Database: Supabase Postgres with the pgvector extension for vector storage and similarity search, plus Supabase Auth (identity) and Supabase Storage (original file bytes) in production.
- Demo fallback: a complete in-memory repository implementation that mirrors the production data-access interface, so the rest of the system is agnostic to which one is active.

8.2 Core Data Flow

Upload: file → parse (PyMuPDF / python-docx / openpyxl / csv) → chunk (~900 chars, page-aware) → embed each chunk (local E5 model) → store chunk + embedding in document_chunks.

Question answering: user question → embed the question → cosine-similarity search over the user's chunks (pgvector) → top passages above a relevance floor → passed as grounding context to the LLM → cited answer returned to the user.

Summarization / action extraction: document's stored pages → grouped into ~6,000-character batches → each batch processed by the LLM independently → partial results merged (and, for actions, de-duplicated) into one final result. See Section 11.2.

Report generation: a report-type-specific focus query (e.g. “risk hazard safety issue critical failure” for a Risk Assessment) is run as a retrieval query against each selected document → the top-scoring passages from each document form the report's source material → the LLM writes the structured report grounded in those passages. See Section 11.3.

9. Data Model

Core entities, as implemented in the production Postgres schema (database/schema.sql). Every table other than profiles carries a user_id and is protected by Row Level Security.

Entity	Key fields	Notes
profiles	id, email, full_name, avatar_url	One row per authenticated user, auto-created on signup.
documents	id, filename, file_type, file_size, status, page_count, chunk_count, error, is_demo	file_type ∈ {pdf, docx, xlsx, csv}; status ∈ {uploading, processing, ready, failed}.
document_chunks	id, document_id, content, page_number, chunk_number, embedding vector(384)	The vector store; embedding is cosine-searched via pgvector's <=> operator.
conversations / messages	title; role, content, sources (jsonb)	sources captures the cited passages (index, filename, page, score) for each assistant message.
actions	title, description, owner, deadline, priority, status, source_document_id	priority ∈ {low, medium, high, critical}; status ∈ {pending, in_progress, completed}.
reports	title, report_type, content,	report_type is one of the six types in

	source_documents, source_filenames	Section 6.6.
analytics	kind, value	Append-only event log (documents_processed, questions_answered, actions_extracted, reports_generated, minutes_saved) aggregated for the dashboard.

10. API Surface (Summary)

All routes below are served under the FastAPI application's /api/v1 prefix (except /health), and require an authenticated user (or the demo stand-in user in demo mode).

Method & Path	Purpose
GET /health	Liveness/config probe — reports demo mode and whether Groq/Supabase are configured.
POST /documents/upload	Upload and process a document.
GET /documents	List the workspace's documents.
GET /documents/{id}/content	Page-by-page extracted text.
DELETE /documents/{id}	Delete a document and its chunks.
POST /documents/{id}/summarize	AI summary (map-reduce).
POST /documents/{id}/extract-actions	AI action extraction (map-reduce, de-duplicated).
POST /chat	Ask a question; returns a grounded, cited answer.
GET /conversations	List conversations.
GET /conversations/{id}	A conversation with full message history.
GET /actions	List all actions.
POST /actions	Create an action manually.
POST /actions/bulk	Create many actions at once.
PATCH /actions/{id}	Update an action.
POST /actions/{id}/communication	Draft an email/WhatsApp message for an action.
GET /reports	List generated reports.
POST /reports/generate	Generate a new report (retrieval-grounded).
GET /analytics	Workspace analytics/KPIs.
GET /usage	Current usage vs. daily/workspace limits.
POST /demo/seed	Seed a populated demo workspace.
POST /demo/reset	Reset the demo workspace.
POST /files/analyze	One-off spreadsheet analysis (not persisted).

11. AI / RAG Pipeline Design

This section documents the retrieval and generation design in detail, since it is the product's core differentiator and the subject of a recent revision from an initial, simpler implementation.

11.1 Embeddings

Document chunks and user questions are embedded with `intfloat/multilingual-e5-small`, an MIT-licensed, 384-dimensional, 94-language sentence embedding model, run locally inside the FastAPI process (no external embedding API call, no per-call cost). Per the model's training convention, chunks are embedded with a “`passage:`” prefix and questions with a “`query:`” prefix; the two use different helper functions (`embed_passages` / `embed_query`) so callers cannot mix them up. This replaced an earlier hashed bag-of-words scheme that had no real semantic understanding — the current model was verified to separate a relevant passage from an unrelated one by a measurable cosine-similarity margin (≈ 0.88 vs. ≈ 0.69 in spot-testing) and to match questions to passages that share no exact keywords with them.

11.2 Map-Reduce Document Processing

Summarization and action extraction process the entire document rather than only its first few thousand characters. A document's stored pages are packed into sequential batches of roughly 6,000 characters each; each batch is summarized (or scanned for actions) independently (the “map” step); for summaries, the partial results are then merged by a further LLM call into one coherent summary (the “reduce” step); for actions, partial results are concatenated and near-duplicate titles (common when the same action is mentioned in more than one section) are collapsed by word-overlap similarity. This ensures an action item or a key fact on, say, page nine of a document is not silently dropped.

11.3 Retrieval-Based Report Generation

Report generation no longer sends a blind, truncated slice of each document's raw text to the LLM. Instead, each report type is associated with a focus query reflecting what that report type cares about (for example, a Risk Assessment's focus query targets “risk hazard safety issue critical failure likelihood impact”). That query is run as a vector-search retrieval against each selected document, and the top-scoring passages — not just the document's opening section — become the report's grounding material, with page citations preserved.

11.4 Vector Storage & Retrieval

In production, chunk embeddings are stored in a Postgres `vector(384)` column (`pgvector` extension) with an `ivfflat` approximate-nearest-neighbour index for cosine distance. Retrieval is performed via a SQL function (`match_document_chunks`) that scopes results to the requesting user and, optionally, to a chosen subset of documents, ordering by cosine similarity. In demo mode, the same interface is served by an in-memory Python implementation computing cosine similarity directly, so the rest of the system does not need to know which backend is active.

11.5 Grounding & Fallback Behaviour

The chat system prompt instructs the model to answer strictly from the retrieved passages, cite them inline, and say so explicitly when the context is insufficient, rather than guessing. If no retrieved passage clears the relevance floor, the system returns a clear “couldn't find enough information” response instead of calling the LLM at all. If the Groq API is not configured, or a live call fails after retries, every AI feature falls back to a deterministic, non-LLM heuristic that still only draws from the retrieved/parsed content — the product never fabricates an answer with no source.

12. Success Metrics / KPIs

- Documents processed successfully (status = ready) as a share of all upload attempts.
- Grounded-answer rate: share of Copilot questions answered with a grounded (cited) response vs. “couldn't find enough information.”
- Estimated minutes saved (tracked per action: document processing, summarization, action extraction, report generation), rolled up on the Analytics dashboard.
- Number of actions extracted and subsequently updated (an indicator that extracted actions are actually being used, not ignored).
- Reports generated per active workspace per week.

13. Constraints, Assumptions & Dependencies

13.1 Constraints

- Maximum upload size: 10 MB per file.
- Demo-protection limits: 5 documents per workspace, 20 chat questions/day, 5 reports/day (configurable via environment variables; intended to bound cost/abuse on a public demo deployment, not a hard product ceiling).
- Only PDF, DOCX, XLSX, and CSV are supported input formats.
- The LLM (Groq) has a request timeout of 60 seconds per call; multi-batch map-reduce operations make multiple sequential calls and take longer for very long documents.

13.2 Assumptions

- Users upload text-extractable documents (e.g. scanned image-only PDFs with no embedded text layer are not supported — parsing requires extractable text).
- A single Groq-hosted model (openai/gpt-oss-120b) is sufficient quality for both grounded Q&A and structured JSON extraction tasks.

- The relevance threshold used to decide whether a retrieved passage is “good enough” to ground an answer needs to be tuned against real usage data over time, and is not a fixed, final value.

13.3 Dependencies

- Groq API availability and API key for all live LLM features.
- Supabase (Postgres + pgvector, Auth, Storage) for the production deployment path; the product degrades gracefully to a fully functional in-memory mode if Supabase is not configured.
- sentence-transformers / PyTorch runtime for local embedding inference on the backend host.

14. Risks & Mitigations

Risk	Mitigation
LLM API outage or rate-limiting interrupts core features.	Automatic retry with exponential backoff on transient errors; deterministic non-LLM fallback responses so the product remains usable (with reduced quality) even if Groq is fully unavailable.
Relevance threshold poorly tuned → either irrelevant passages ground an answer, or relevant ones are wrongly excluded.	Threshold was set from empirical spot-testing of the current embedding model's similarity distribution and is flagged in code as needing further calibration against real queries before being considered final.
Switching the embedding model in the future breaks existing stored vectors (different models are not comparable).	A documented migration path (schema migration script + re-embedding script) exists for exactly this scenario, having already been exercised once during this build.
Long documents exceed a single LLM call's context budget.	Map-reduce processing (Section 11.2) and retrieval-based report generation (Section 11.3) ensure the whole document is considered, not just its opening section.
Demo-mode data loss on backend restart (in-memory only).	Documented and intentional trade-off for frictionless evaluation; production deployments use persistent Supabase storage instead.

15. Release Status & Open Items

Current state: a functionally complete MVP, runnable end-to-end in demo mode with no external database. All features in Section 6 are implemented and have been smoke-tested against live Groq calls and the current embedding model.

15.1 Open items before a production launch

- Provision and configure a production Supabase project (Postgres + pgvector, Auth, Storage) and apply database/schema.sql, functions.sql, indexes.sql, and policies.sql.
- Calibrate the retrieval relevance threshold against a real corpus of documents and representative user questions.
- Load-test map-reduce document processing on genuinely long documents (50+ pages) to validate latency and Groq rate-limit headroom.
- Decide on and implement real usage-limit tiers (the current limits are demo-protection defaults, not a pricing/plan model).

16. Appendix: Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — grounding an LLM's response in passages retrieved from a knowledge store, rather than the model's own memorized knowledge.
Embedding	A numeric vector representation of text such that semantically similar text has a similar vector; used to power similarity search.
pgvector	A Postgres extension adding a vector data type and similarity-search operators/indexes.
Chunk	A sub-section of a document (here, ~900 characters) stored and embedded independently so retrieval can return precise, cited passages instead of whole documents.
Map-reduce (in this document)	Processing a long document by handling it in independent sections (“map”) and then combining those partial results into one final output (“reduce”).
Grounded answer	An answer the LLM produced using only the retrieved source passages, with citations back to them.
Demo mode	A configuration in which the product runs against an in-memory data store and a stand-in user, requiring no external database or authentication setup.